

串口通信

串口通信

一、学习目标

串口通讯

串口工作模式

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. 串口时钟配置

3. 串口参数配置

4. 写入程序

5. 编译

四、程序分析

五、实验现象

一、学习目标

1.学习MSPM0G3507主板串口的基本使用。

2.了解串口通讯基本知识。

串口通讯

串口通信(Serial Communication)，是指外设和计算机间通过数据信号线、地线等按位进行传输数据的一种通信方式，属于串行通信方式。

串口通信的关键参数包括波特率、数据位、停止位和奇偶校验位。为了确保通信正常，这些参数必须在通信双方保持一致。

串口工作模式

1. 单工模式 (Simplex)

- 特点: 数据只能单向传输，发送端负责发送，接收端负责接收，没有反向通信。
- 应用: 适用于只需单向传递信息的场景，如传感器的数据输出。

2. 全双工模式 (Full-Duplex)

- 特点: 数据可以同时两个方向上传输，发送和接收互不干扰。
- 实现: 需要两条数据线，分别用于发送 (TX) 和接收 (RX)。
- 应用: 常用于需要高效、实时通信的场景，如计算机与外设的交互。

3. 半双工模式 (Half-Duplex)

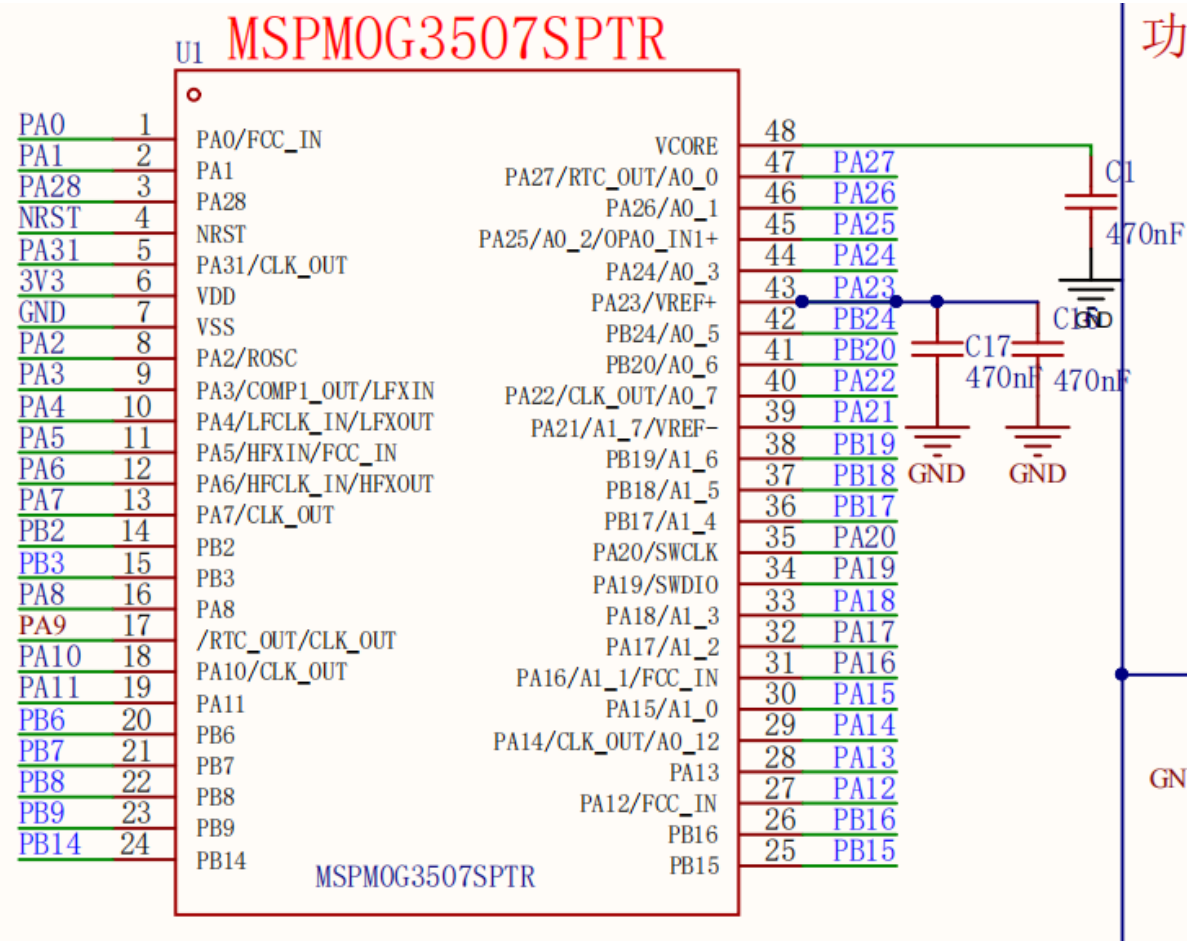
- 特点: 数据在两个方向上传输，但不能同时进行，必须交替发送和接收。
- 实现: 发送和接收共享一条通信线，依靠时序切换方向。
- 应用: 适用于对实时性要求不高的场景，如对讲机通信。

这三种模式适应了不同通信需求，可根据具体应用场景灵活选择。

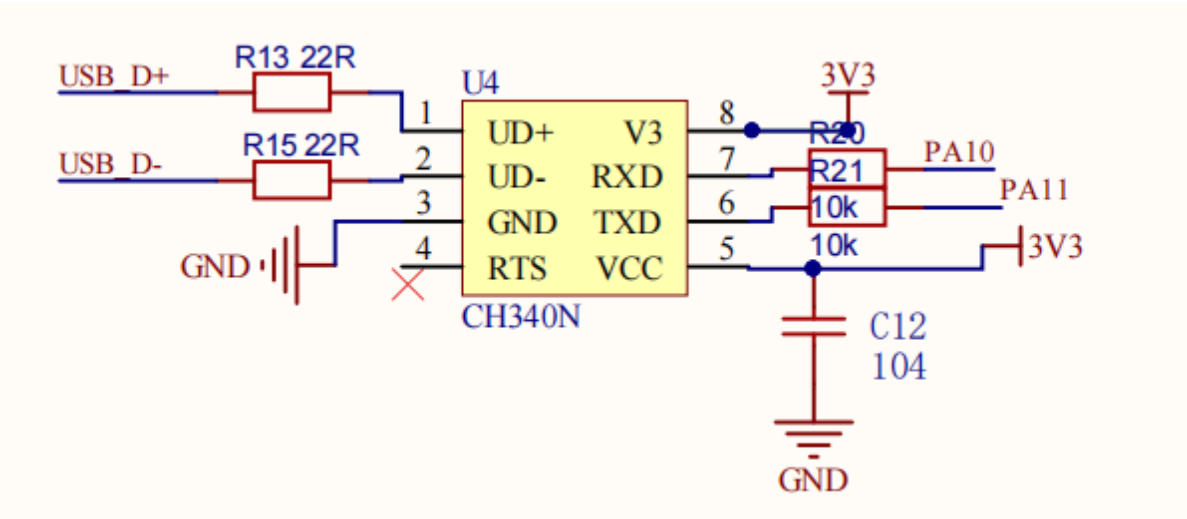
二、硬件搭建

由于开发板上自带CH340串口电路，可直接通过USB线实现串口通信，无需外接USB转TTL模块。

MSPM0G3507主控图



Type-C部分原理图



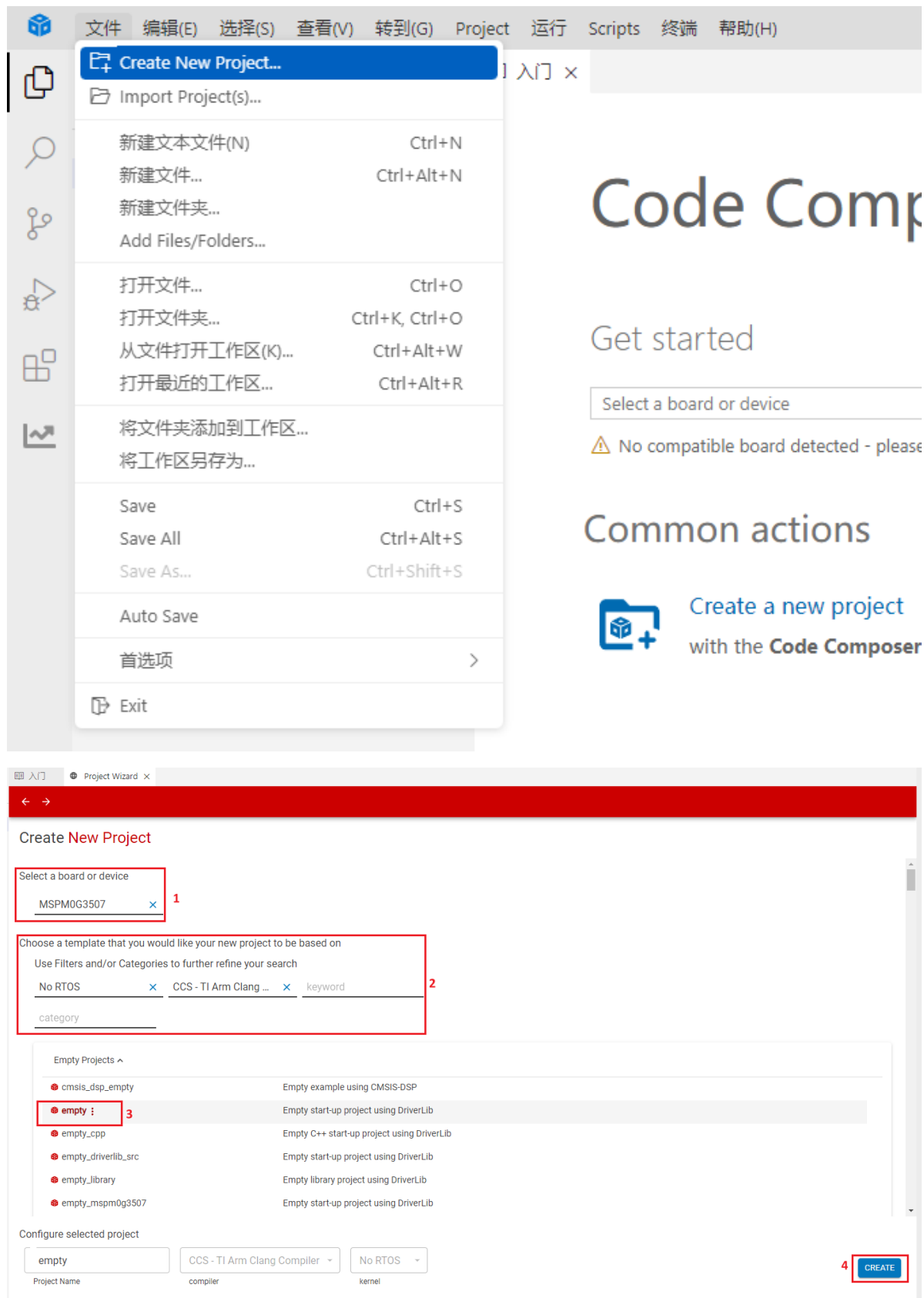
21	PA10	UART0_TX [2] / SPI0_POC1 [3] / I2C0_SDA [4] / TIMA1_C0 [5] / TIMG12_C0 [6] / TIMA0_C2 [7] / I2C1_SDA [8] / CLK_OUT [9]	56	18	14	15	高驱动
22	PA11	UART0_RX [2] / SPI0_SCK [3] / I2C0_SCL [4] / TIMA1_C1 [5] / COMP0_OUT [6] / TIMA0_C2N [7] / I2C1_SCL [8]	57	19	15	16	高驱动

三、实验步骤

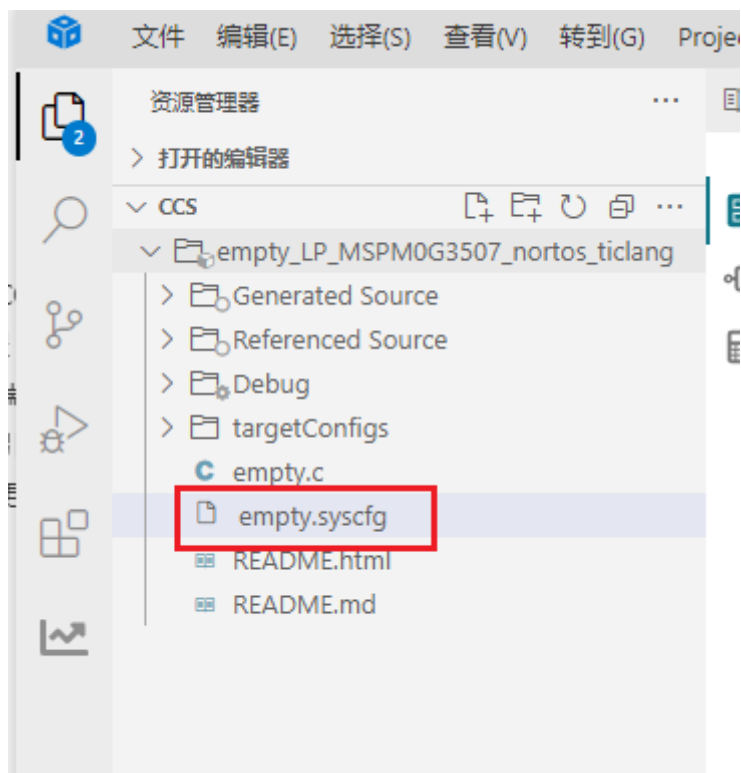
本节课使用PA10和PA11引脚上的UART0外设，搭配开发板板载的CH340这个USB转串口芯片，实现接收上位机发送过来的串口数据，再通过串口发送功能将数据发送给上位机的操作。

1. 打开SYSCONFIG配置工具

在CCS中新建一个空白工程 empty。



在CCS的左侧工作区中找到并打开empty.syscfg文件。



2. 串口时钟配置

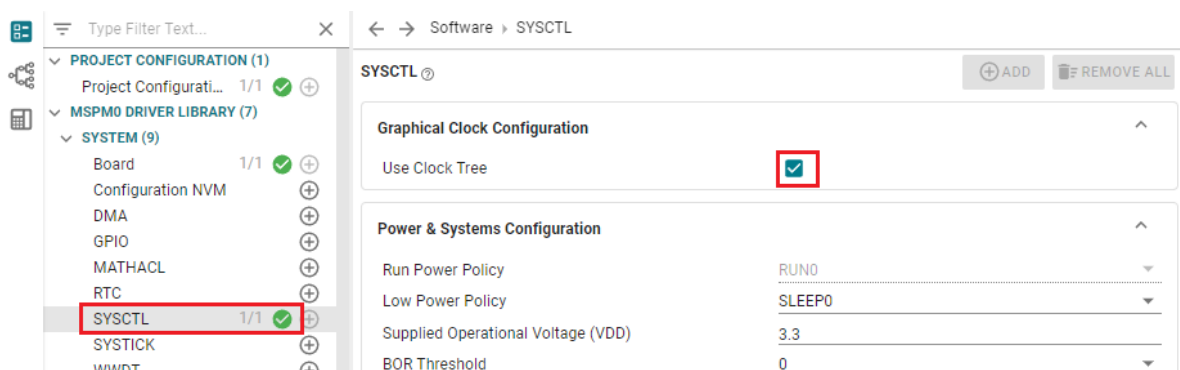
串口的时钟来源一共有三个，分别是：

BUSCLK: 由内部高频振荡器提供的CPU时钟，通常芯片出厂时设置为了32MHz。

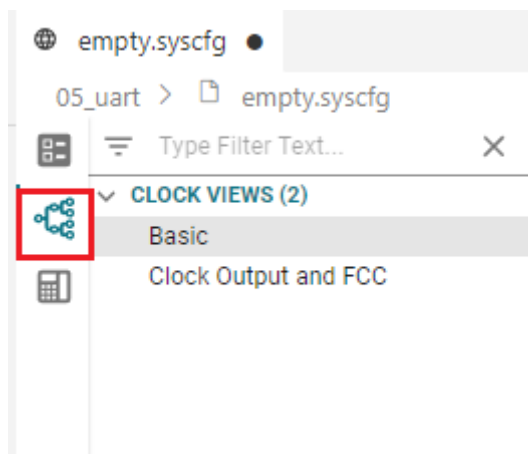
MFCLK: 只能使用固定的4MHz时钟(参考用户手册132页)。开启的话需要配置时钟树的SYSOSC_4M分支，才能够正常开启。

LFCLK: 由内部的低频振荡器提供时钟（32KHz）。在运行、睡眠、停止和待机模式下有效，使用该时钟可以实现更低的功耗。

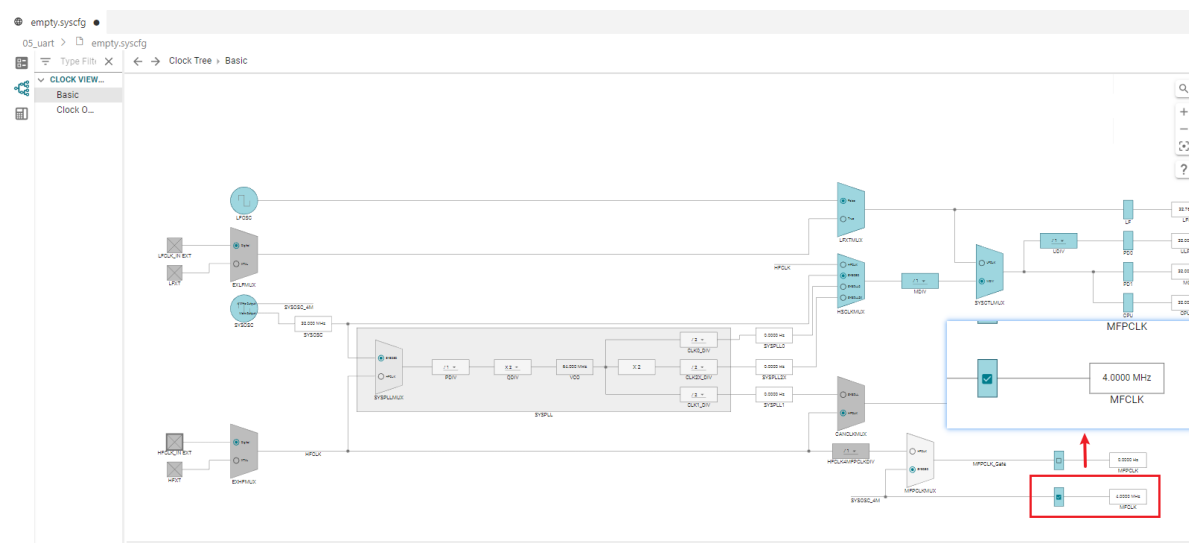
本案例使用的是MFCLK作为串口的时钟来源，开启MFCLK需要在 SYSCONFIG 中配置时钟树。在 SYSCONFIG左侧的选项卡中找到SYSCTL选项，在选项页中找到 Use Clock Tree 进行打勾，以开启时钟树的配置。



点击 SYSCONFIG 界面最左边第二个图标，可以打开时钟树。

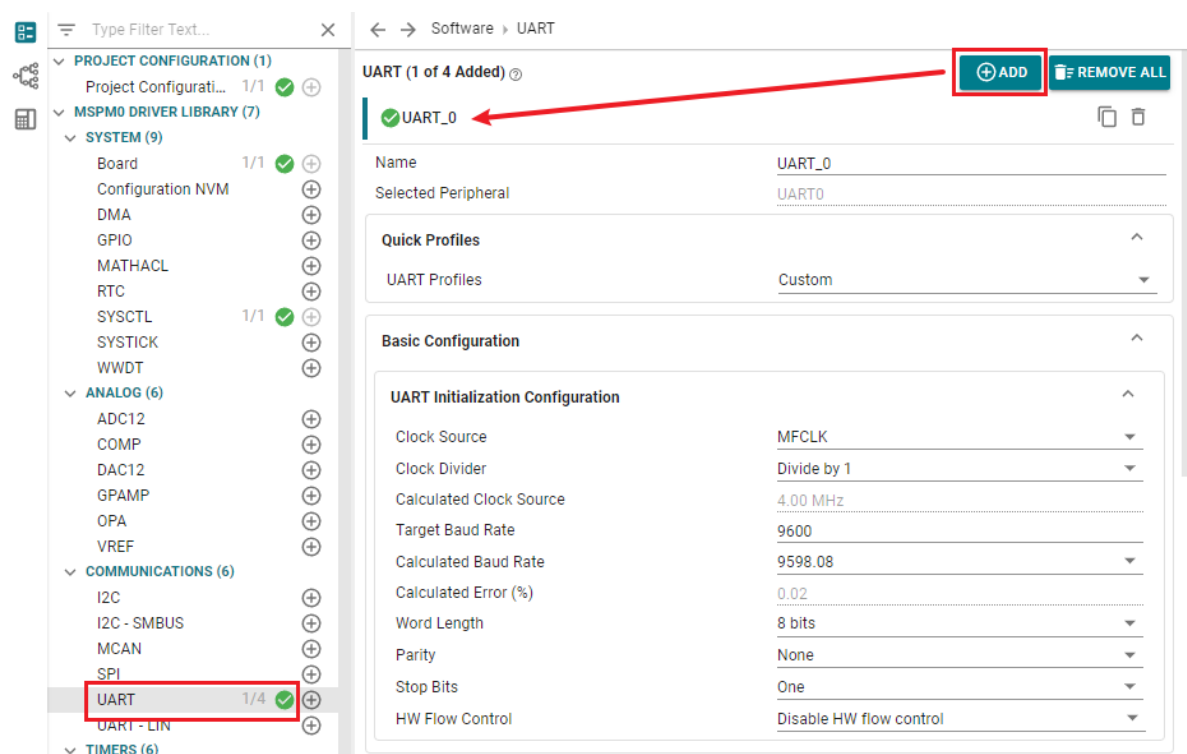


在时钟树中打开MFCLK的开关，如下图所示位置，点击SYSOSC_4M分支上的选项块，开启MFCLK时钟。当右侧出现4.0000Mhz时说明设置完成。



3. 串口参数配置

在 SYSCONFIG 中，左侧选择 MCU 外设，找到 UART 选项并点击进入。在 UART 中点击 ADD，添加一组串口外设。



配置与keil环境案例中一致，这里不再说明。

UART (1 of 4 Added) ⓘ

⊕ ADD

🗑 REMOVE ALL

✓ UART_0

📄 🗑

Name

UART_0

Selected Peripheral

UART0

Quick Profiles

^

UART Profiles

Custom

▼

Basic Configuration

^

UART Initialization Configuration

^

Clock Source

MFCLK

▼

Clock Divider

Divide by 1

▼

Calculated Clock Source

4.00 MHz

Target Baud Rate

9600

Calculated Baud Rate

9598.08

▼

Calculated Error (%)

0.02

Word Length

8 bits

▼

Parity

None

▼

Stop Bits

One

▼

HW Flow Control

Disable HW flow control

▼

Advanced Configuration

^

UART Mode

Normal UART Mode

▼

Communication Direction

TX and RX

▼

Oversampling

16x

▼

Enable FIFOs

☐

Analog Glitch Filter

Disabled

▼

Digital Glitch Filter

0

Calculated Digital Glitch Filter

0.00 s

RX Timeout Interrupt Counts

0

Calculated RX Timeout Interrupt

0.00 s

Enable Internal Loopback

☐

Enable Majority Voting

☐

Enable MSB First

☐

Interrupt Configuration ⓘ

^

Enable Interrupts

Receive

▼

Interrupt Priority

Default

▼

PinMux Peripheral and Pin Configuration

^

UART Peripheral

Any(UART0)

▼

RX Pin

PA11/57

▼

🔒

TX Pin

PA10/56

▼

🔒

通过快捷键 Ctrl+S 将.syscfg文件中的配置保存。

4. 写入程序

配置好串口之后，我们还要手动编写串口的中断服务函数。因为我们开启了串口的接收中断，当串口接收到数据时，就会触发一次中断，触发中断就会执行中断服务函数。每个中断对应的中断服务函数名称通常是固定的，不能随意修改，否则将无法正确进入中断服务函数。中断服务函数的具体名称

UART0_IRQHandler。

也可以在编译后，在我们生成的ti_msp_dl_config.h文件中看到关于串口0中断服务函数的宏定义。

```
empty.syscfg  empty.c  ti_msp_dl_config.h x
05_uart > Debug > ti_msp_dl_config.h > ...
68  */
69
70
71  /* clang-format off */
72
73  #define POWER_STARTUP_DELAY (16)
74
75
76
77  #define CPUCLK_FREQ 32000000
78
79
80
81  /* Defines for UART_0 */
82  #define UART_0_INST UART0
83  #define UART_0_INST_FREQUENCY 4000000
84  #define UART_0_INST_IRQHandler UART0_IRQHandler
85  #define UART_0_INST_INT_IRQN UART0_INT_IRQn
86  #define GPIO_UART_0_RX_PORT GPIOA
87  #define GPIO_UART_0_TX_PORT GPIOA
88  #define GPIO_UART_0_RX_PIN DL_GPIO_PIN_11
89  #define GPIO_UART_0_TX_PIN DL_GPIO_PIN_10
90  #define GPIO_UART_0_IOMUX_RX (IOMUX_PINCM22)
91  #define GPIO_UART_0_IOMUX_TX (IOMUX_PINCM21)
92  #define GPIO_UART_0_IOMUX_RX_FUNC IOMUX_PINCM22_PF_UART0_RX
93  #define GPIO_UART_0_IOMUX_TX_FUNC IOMUX_PINCM21_PF_UART0_TX
94  #define UART_0_BAUD_RATE (9600)
95  #define UART_0_IBRD_4_MHZ_9600_BAUD (26)
96  #define UART_0_FBRD_4_MHZ_9600_BAUD (3)
```

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

volatile unsigned int delay_times = 0;
volatile unsigned char uart_data = 0;

void delay_ms(unsigned int ms);
void uart0_send_char(char ch);
void uart0_send_string(char* str);

int main(void)
{
    SYSCFG_DL_init();
    //清除串口中断标志 clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    //使能串口中断 Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);

    while (1)
    {
```

```

        //LED引脚输出高电平 LED pin outputs high level
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(500);
        //LED引脚输出低电平 LED pin outputs low level
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(500);
    }
}

//搭配滴答定时器实现的精确ms延时 Accurate ms delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

//串口发送单个字符 Send a single character through the serial port
void uart0_send_char(char ch)
{
    //当串口0忙的时候等待，不忙的时候再发送传进来的字符
    // Wait when serial port 0 is busy, and send the incoming characters when it
    is not busy
    while( DL_UART_isBusy(UART_0_INST) == true );
    //发送单个字符 Send a single character
    DL_UART_Main_transmitData(UART_0_INST, ch);
}

//串口发送字符串 Send string via serial port
void uart0_send_string(char* str)
{
    //当前字符串地址不在结尾 并且 字符串首地址不为空
    // The current string address is not at the end and the string first address
    is not empty
    while(*str!=0&&str!=0)
    {
        //发送字符串首地址中的字符，并且在发送完成之后首地址自增
        // Send the characters in the first address of the string, and increment
        the first address after sending.
        uart0_send_char(*str++);
    }
}

//滴答定时器的中断服务函数 Tick timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

//串口的中断服务函数 Serial port interrupt service function
void UART_0_INST_IRQHandler(void)
{
    //如果产生了串口中断 If a serial port interrupt occurs
    switch( DL_UART_getPendingInterrupt(UART_0_INST) )
    {
        case DL_UART_IIDX_RX://如果是接收中断 If it is a receive interrupt
            //发送过来的数据保存在变量中 The data sent is saved in the variable
            uart_data = DL_UART_Main_receivedData(UART_0_INST);
    }
}

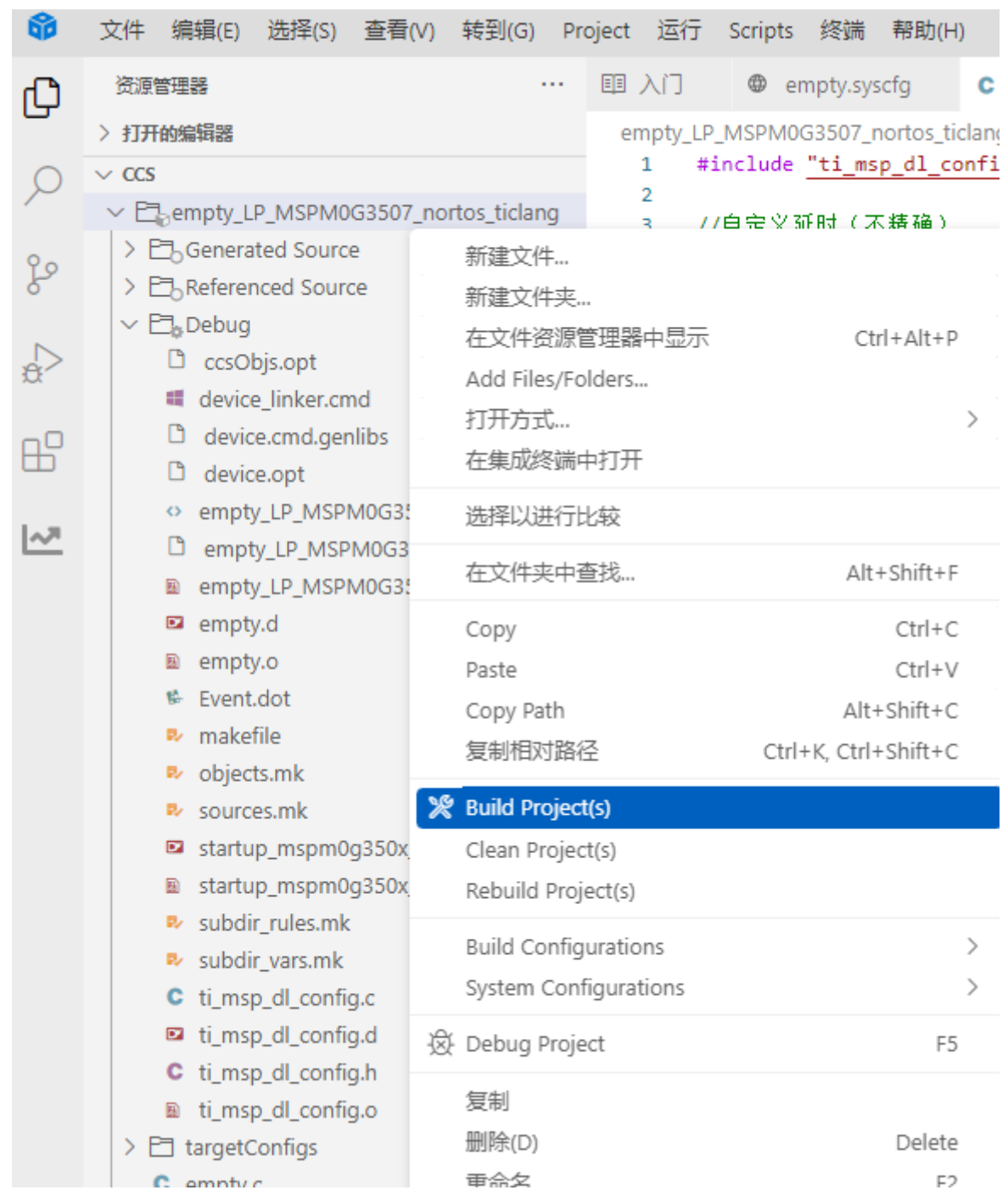
```



```
//将保存的数据再发送出去 Send the saved data again
uart0_send_char(uart_data);
break;

default://其他的串口中断 Other serial port interrupts
break;
}
}
```

5. 编译



编译成功了，即可将程序下载到开发板。

四、程序分析

- empty.c

```

10  int main(void)
11  {
12      SYSCFG_DL_init();
13      //清除串口中断标志 Clear the serial port interrupt flag
14      NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
15      //使能串口中断 Enable serial port interrupt
16      NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
17
18      while (1)
19      {
20          //LED引脚输出高电平 LED pin outputs high level
21          DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
22          delay_ms(500);
23          //LED引脚输出低电平 LED pin outputs low level
24          DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
25          delay_ms(500);
26      }
27  }

```

串口中断必须手动开启。函数 `NVIC_EnableIRQ` 指定开启某一个中断。开启之前要先清除中断标志位，否则开启中断后将会自动进入中断。

```

35  //串口发送单个字符 Send a single character through the serial port
36  void uart0_send_char(char ch)
37  {
38      //当串口0忙的时候等待，不忙的时候再发送传进来的字符
39      // Wait when serial port 0 is busy, and send the incoming characters when it is not busy
40      while( DL_UART_isBusy(UART_0_INST) == true );
41      //发送单个字符 Send a single character
42      DL_UART_Main_transmitData(UART_0_INST, ch);
43  }
44  //串口发送字符串 Send string via serial port
45  void uart0_send_string(char* str)
46  {
47      //当前字符串地址不在结尾 并且 字符串首地址不为空
48      // The current string address is not at the end and the string first address is not empty
49      while(*str!=0&&str!=0)
50      {
51          //发送字符串首地址中的字符，并且在发送完成之后首地址自增
52          // Send the characters in the first address of the string, and increment the first address after sending.
53          uart0_send_char(*str++);
54      }
55  }

```

定义两个函数，`uart0_send_char` 函数通过串口发送单个字符，`uart0_send_string` 函数通过串口发送字符串。

```

66  //串口的中断服务函数 Serial port interrupt service function
67  void UART_0_INST_IRQHandler(void)
68  {
69      //如果产生了串口中断 If a serial port interrupt occurs
70      switch( DL_UART_getPendingInterrupt(UART_0_INST) )
71      {
72          case DL_UART_IIDX_RX://如果是接收中断 If it is a receive interrupt
73              //接发送过来的数据保存在变量中 The data sent is saved in the variable
74              uart_data = DL_UART_Main_receiveData(UART_0_INST);
75              //将保存的数据再发送出去
76              uart0_send_char(uart_data);
77              break;
78
79          default://其他的串口中断 Other serial port interrupts
80              break;
81      }
82  }

```

UART_0_INST_IRQHandler 函数实现串口接收数据并回传的功能。通过 DL_UART_getPendingInterrupt() 获取当前 UART0 的中断类型，如果接收到了数据触发的UART 中断，就将接收到的数据保存在变量中，之后再发出去。

五、实验现象

程序下载完成之后，可以通过串口调试助手发送数据给开发板，就能看到开发板返回的数据。

